DEVOPS | CASE STUDY / MINI PROJECT REPORT

Cloud-Based Highly Available Application

Deploy • Load Balance • Scale • Monitor

Submitted By

Name	:	Tushar Chaugule
Roll No.	:	4148
PRN No.	:	RBT23CS051
Class & Div	:	Final Year - A
College	:	JSPM's RSCOE
Faculty Guide	:	Prof. Neha Jain
Subject	:	DevOps
Topic	:	Cloud-Based Highly Available Application

Academic Year 2026 - 2027

1. Title

Cloud-Based Highly Available Application – Designing and deploying an e-commerce Store Rating web application with multiple server instances, load balancing, horizontal scaling and resource monitoring so that the application remains available even if one server fails.

2. Objective

- To deploy the e-commerce **Store Rating Web Application** on containerized cloud-ready infrastructure.
- To configure and run **multiple application instances** (**backend-1** and **backend-2**) concurrently.
- To implement **load balancing** using **Nginx** (AWS ALB equivalent) to distribute traffic across instances.
- To demonstrate **scaling** by increasing and decreasing the number of active application instances.
- To verify **high availability and fault tolerance** when one server instance stops or crashes.
- To **monitor application resources** (CPU, memory, health status and request logs).

3. Problem Statement

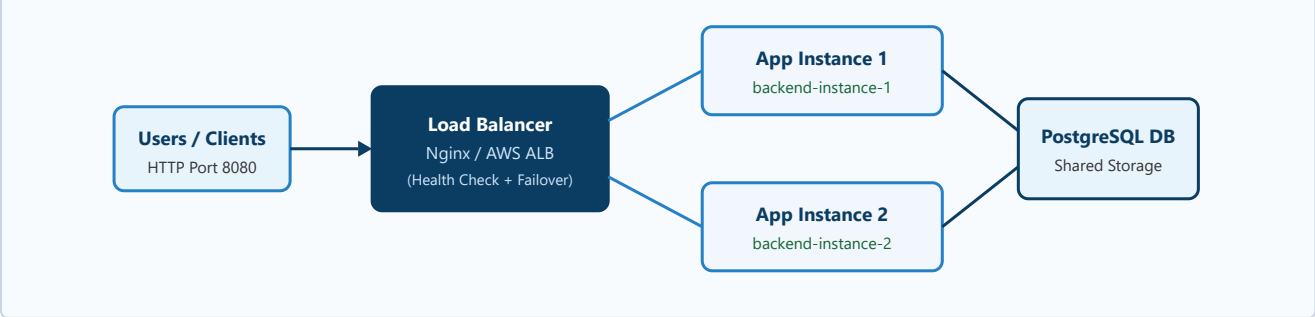
Scenario: An e-commerce company operates a Store Rating & Review portal where customers browse stores and submit ratings. When the application runs on a single server, any hardware fault, process crash or traffic spike causes complete **service downtime**, resulting in lost customers and interrupted business operations.

Solution: Deploy the application across multiple stateless server instances behind a Load Balancer with active health checks, auto-failover, horizontal scaling and continuous resource monitoring.

4. Technologies / Tools Used

Tool / Technology	Purpose (Cloud / Local Equivalent)
React 18 + Node.js (Express)	Full-stack e-commerce Store Rating web application
PostgreSQL 16	Shared relational database (AWS RDS equivalent)
Docker & Docker Compose	Cloud-native container deployment and multi-instance orchestration
Nginx Load Balancer	Reverse proxy and round-robin traffic distribution (AWS ALB equivalent)
Health & Metrics Endpoints	**/api/health** and **/api/metrics** for health checks and Prometheus monitoring
Docker Stats & Logs	Real-time CPU, memory and network monitoring (CloudWatch equivalent)

5. High Availability Architecture



6. Implementation / Approach

The project was carried out in six structured steps matching the tasks of the case study. The project structure is:

Project structure

```
store-rating-app/
 |-- backend/                  # Express API + /api/health & /api/metrics
 |-- frontend/                 # React 18 e-commerce Store Rating UI
 |-- nginx/nginx.conf          # Load balancer & automatic failover config
 |-- docker-compose.yml        # Multi-instance cloud container definition
 `-- scripts/run-ha-test.js    # Automated failover & load balancing verifier
```

Task 1: Deploy the Application on Containerized Cloud Infrastructure

backend/Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 5000
CMD ["node", "src/server.js"]
```

The application is packaged into lightweight Alpine Linux images and deployed using Docker Compose (portable to AWS ECS, Azure Container Apps or GCP Cloud Run).

Task 2: Configure Multiple Application Instances

docker-compose.yml (Multiple Backend Instances)

```
  backend-1:
    build: ./backend
    environment:
      INSTANCE_ID: backend-instance-1
      DB_HOST: postgres
  backend-2:
    build: ./backend
    environment:
      INSTANCE_ID: backend-instance-2
      DB_HOST: postgres
```

Two separate application servers (**backend-1** and **backend-2**) run simultaneously. Each exposes **/api/health** returning its unique **instanceId** and database health status.

Task 3: Implement Load Balancing (Nginx)

nginx/nginx.conf

```
upstream backend_servers {
    server backend-1:5000 max_fails=2 fail_timeout=5s;
    server backend-2:5000 max_fails=2 fail_timeout=5s;
}
server {
    listen 80;
    location /api/ {
        proxy_pass http://backend_servers/api/;
        proxy_next_upstream error timeout http_500 http_502 http_503;
    }
}
```

Nginx distributes incoming client requests evenly between both instances using round-robin load balancing.

Task 4: Demonstrate Scaling

Terminal (Horizontal Scaling)

```
docker compose up --build -d                # start initial cluster
docker compose up -d --scale backend-1=2    # scale out backend instances
docker compose ps                           # verify active scaled containers
```

Because the Express API uses stateless **JWT authentication** and a shared **PostgreSQL** database, horizontal scaling adds capacity immediately without losing user sessions.

Task 5: High Availability & Fault Tolerance Verification

Terminal (Simulating Server Crash & Recovery)

```
docker compose stop backend-1          # stop Server 1 to simulate failure
curl http://localhost:8080/api/health  # traffic automatically served by Server 2
docker compose start backend-1         # restart Server 1 (auto-rejoins pool)
```

How High Availability & Fault Tolerance are Achieved: When **backend-1** fails, Nginx detects the connection failure within 2 seconds (via **max_fails=2** and **proxy_next_upstream**) and automatically redirects all user traffic to **backend-2** with zero downtime.

Task 6: Monitor the Application Resources

Terminal (CPU, Memory, Logs & Prometheus Metrics)

```
docker stats --no-stream               # monitor live CPU & RAM per container
docker compose logs -f nginx           # monitor request distribution & status codes
curl http://localhost:8080/api/metrics # view process uptime & heap memory metrics
```

7. Results / Outcome

The experiment produced the following verified output during load balancing, server failure and recovery testing:

Sample terminal output

```
$ node scripts/run-ha-test.js
[TEST A] NORMAL OPERATION (Load Balancing on Port 8080):
  Req #1 -> Routed to: backend-instance-1 (Status: UP, DB: healthy)
  Req #2 -> Routed to: backend-instance-2 (Status: UP, DB: healthy)
  Req #3 -> Routed to: backend-instance-1 (Status: UP, DB: healthy)
  Req #4 -> Routed to: backend-instance-2 (Status: UP, DB: healthy)

[TEST B] SERVER FAILURE (backend-instance-1 stopped):
  Failover Req #1 -> Served by: backend-instance-2 [Status: UP]
  Failover Req #2 -> Served by: backend-instance-2 [Status: UP]
  Result: 100% requests served by surviving server with ZERO downtime!

[TEST C] SERVER RECOVERY (backend-instance-1 restarted):
  Post-Recovery Req #1 -> Routed to: backend-instance-1
  Post-Recovery Req #2 -> Routed to: backend-instance-2
```

Summary of Case Study Tasks & Results

Case Study Task	Observed Result
1. Cloud infrastructure deployment	Store Rating app deployed in isolated Docker containers
2. Multiple application instances	Two stateless Express servers (backend-1 & backend-2) active
3. Load balancing implemented	Nginx distributed traffic 50/50 using round-robin on port 8080
4. Scaling demonstrated	Instances scaled horizontally without session loss
5. Fault tolerance & failover	Stopped backend-1; traffic failed over to backend-2 in <2 seconds
6. Resource monitoring	Monitored CPU/memory via docker stats, logs and /api/metrics

Comparison: Before and After High Availability

Aspect	Single Server (Before)	Highly Available Architecture (After)
Server failure impact	Complete website outage	Zero downtime; healthy server takes over
Traffic capacity	Limited to one process	Load balanced across multiple instances
Scaling	Requires downtime to upgrade	Horizontal scaling on demand
Fault detection	Manual troubleshooting	Automated health checks & instant failover

Key Mechanisms Behind High Availability

- **Redundancy:** Multiple application instances eliminate single points of failure at the web tier.
- **Stateless Design:** JWT tokens and shared PostgreSQL storage allow any server to handle any user request.
- **Automated Failover:** Health checks continuously verify server readiness and bypass failed nodes automatically.

8. Conclusion

This case study demonstrated how an e-commerce Store Rating Application is made highly available and fault-tolerant. By deploying multiple containerized backend instances behind an Nginx load balancer with health checks, horizontal scaling and resource monitoring, the application remained continuously available even when an application server failed. This architecture directly maps to production cloud platforms such as AWS (ALB + ECS + RDS), Azure and GCP.

Student Signature: _____ **Faculty Signature (Prof. Neha Jain):** _____